- *Response handling*—Instead of streaming every token, this example transforms results into structured UI messages and displays them in a richer card component.

12.5.2 Building an end-to-end integration with LangChain.js

In this step, we turn to the backend API route. The MCP server implementation remains the same as before—it still exposes the `get-norris-joke` tool backed by the Chuck Norris API, so there is no need to change it. Our focus now is on the Next.js API route that connects the frontend to LangChain, Gemini, and the MCP server.

Figure 12.7 shows the overall flow of messages through the system. The API route sits in the middle of that flow. Its job is to receive the user's input from the frontend, pass it into a LangChain agent that can use MCP tools, and then stream the agent's output back to the user interface. We will build this in four parts:

1. Create an MCP client to connect to the Chuck Norris MCP server and discover its tools.
2. Initialize the Gemini model.
3. Configure a LangChain React agent that combines the model with the MCP tools.
4. Invoke the agent with the user's message, close the MCP client, and stream the result back to the UI.

Listing 12.3 shows how this comes together in a simplified version of the API route. The full implementation in the repository includes extra helpers for normalizing messages and handling streaming, but here we focus on the core ideas.

Listing 12.3 Next.js API route with LangChain and MCP

```
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { createUIMessageStreamResponse } from "ai";
import { MultiServerMCPClient } from "@langchain/mcp-adapters";
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { HumanMessage } from "@langchain/core/messages";

async function createMCPClient() {
  const client = new MultiServerMCPClient({
    useStandardContentBlocks: true,
    mcpServers: {
      norris: {
        transport: "stdio",
        command: "node",
        args: ["app/stdio/server.js"],
        restart: { enabled: true, maxAttempts: 3, delayMs: 1000 },
      },
    },
  });
  const tools = await client.getTools();
  return { client, tools };
}
```

Creates a MultiServerMCPClient to connect to the Chuck Norris MCP server and fetch available tools